

FlowNet sur em780 (Python 3.12.3)

I. Installation du projet et des librairies

Copie du projet FlowNet :

- Copie du code dans le répertoire « FlowNetPytorch » :
Cloner le projet depuis
<https://github.com/paul-pp/flownet>
Copie du modèle pré-entraîné dans le répertoire « trained_model » :
Télécharger un des fichiers (depuis le répertoire Pytorch) .pth avec ce lien
https://drive.google.com/drive/folders/16eo3p9dO_vmssxRoZCmWkTpNjKRzJzn5

Créer un répertoire « emtest » :

Dans ce répertoire, il faut copier des paires d'images. Pour cela, il faut que les 2 images d'une paire proviennent d'une même vidéo (séparées d'un petit intervalle de temps) et soient de même dimension. Celles-ci doivent être nommées de la manière suivante : <nom_de_la_paireA>1.<extension>, <nom_de_la_paireA>2.<extension>, <nom_de_la_paireB>1.<extension>, <nom_de_la_paireB>2.<extension>,...

Copie du projet sur le front : (en local)

- Copie du code :
`scp -r FlowNetPytorch front.mono.lip6.ext:~/`
- Copie d'un modèle pré-entraîné :
`scp -r trained_model front.mono.lip6.ext:~/`
- Copie des images dont on cherche le flux optique :
`scp -r emtest front.mono.lip6.ext:~/`

Connexion à la machine em780 : (en local)

`ssh em780.mono.lip6.ext`

Déplacement du projet sur scratch : (depuis /nfs/users/<username>-nfs)

`mv FlowNetPytorch /scratch/<username>-nfs/code_source_pytorch/`

`mv trained_model/ /scratch/<username>-nfs/code_source_pytorch/`

`mv emtest /scratch/<username>-nfs/images_tests/`

Création d'un environnement python et activation: (depuis /scratch/<username>-nfs/code_source_pytorch)

- Création :
`python3 -m venv venv-flownet`

- Activation :
source venv-flownet/bin/activate

Installation des librairies nécessaires dans l'environnement : (depuis /scratch/<username>-nfs/code_source_pytorch et lorsque «venv-flownet» est activé)

```
pip3 install -r FlowNetPytorch/requirements.txt
```

Visualisation des versions des librairies avec « pip3 list »:

Package	Version
-----	-----
absl-py	2.1.0
filelock	3.16.1
fsspec	2024.10.0
grpcio	1.68.0
imageio	2.36.0
Jinja2	3.1.4
Markdown	3.7
MarkupSafe	3.0.2
mpmath	1.3.0
networkx	3.4.2
numpy	2.1.3
nvidia-cublas-cu12	12.4.5.8
nvidia-cuda-cupti-cu12	12.4.127
nvidia-cuda-nvrtc-cu12	12.4.127
nvidia-cuda-runtime-cu12	12.4.127
nvidia-cudnn-cu12	9.1.0.70
nvidia-cufft-cu12	11.2.1.3
nvidia-curand-cu12	10.3.5.147
nvidia-cusolver-cu12	11.6.1.9
nvidia-cuspars-cu12	12.3.1.170
nvidia-nccl-cu12	2.21.5
nvidia-nvjitlink-cu12	12.4.127
nvidia-nvtx-cu12	12.4.127
packaging	24.2
path	17.0.0
pillow	11.0.0
pip	24.0
protobuf	5.28.3

scipy	1.14.1
setuptools	75.5.0
six	1.16.0
spatial_correlation_sampler	0.5.0
sympy	1.13.1
tensorboard	2.18.0
tensorboard-data-server	0.7.2
torch	2.5.1
torchvision	0.20.1
tqdm	4.67.0
triton	3.1.0
typing_extensions	4.12.2
Werkzeug	3.1.3

Désactiver l'environnement virtuel :

\$ deactivate

II. Lancement de l'inférence

Activer l'environnement «venv-flownet» :(depuis /scratch/<username>-nfs/code_source_pytorch)
source venv-flownet/bin/activate

Lancer l'inférence : (depuis /scratch/<username>-nfs/code_source_pytorch)

```
python3 FlowNetPytorch/run_inference.py /scratch/<username>-nfs/images_tests/emtest trained_model/flownets_EPE1.951.pth
```

Remarques :

- Le 1er chemin correspond au script python permettant l'inférence du modèle pré-entraîné situé au 3^e chemin. Les images en entrée sont situées dans la 2^e chemin.
- Si vous tentez de lancer la commande précédente dans un répertoire qui ne vous appartient pas il est probable que vous ayez une erreur (de droit d'écriture dans le dossier d'un utilisateur externe) car le programme doit enregistrer l'image de flux optique dans le dossier « emtest » qui appartient à l'utilisateur ayant le code source. Pour éviter cela, il faudrait que vous copiez le dossier emtest dans votre répertoire avec la paire d'images à tester. Et vous pourrez relancer la commande suivante :

```
python3 FlowNetPytorch/run_inference.py /scratch/<votre_username>-nfs/images_tests/emtest trained_model/ flownets_EPE1.951.pth
```

- L'option -g (--gpu) (et respectivement -c (--cpu)) permet de forcer le lancement de FlowNet sur le gpu (respectivement sur le cpu). Par défaut, si le gpu existe, l'inférence sera lancée sur gpu. Un exemple de commande pour lancer sur cpu uniquement pourrait être :

```
python3 FlowNetPytorch/run_inference.py /scratch/<username>-nfs/images_tests/emtest trained_model/ flownets_EPE1.951.pth -c
```

- L'option -v (--output-value) permet de spécifier le type de sortie pour le flot optique désiré. Il y'a 2 types de sorties possibles :
 - o le type «raw» est un fichier de sortie de type «.npy » qui correspond à un tableau stockant le flot optique (de 2 frames successives)
 - o le type «viz» est une image (de format png par exemple) qui correspond à la représentation en couleur du flot optique entre 2 frames successives.

Enfin si vous spécifiez «both», le programme génère automatiquement les types «raw» et «viz» simultanément (option par défaut). Voici un exemple de commande pour générer que des visualisations en couleur du flot optique :

```
python3 FlowNetPytorch/run_inference.py /scratch/<username>-nfs/images_tests/emtest trained_model/ flownets_EPE1.951.pth -v viz
```

- Si plusieurs paires d'images sont présentes dans le dossier emtest, run_inference.py les traitera toutes.

On obtient la sortie suivante pour cpu :

```
"=> will save raw output and RGB visualization
=> fetching img pairs in '/scratch/poupeaup-nfs/images_tests/emtest'
=> will save everything to /scratch/poupeaup-nfs/images_tests/emtest/flow
1 samples found

/scratch/poupeaup-nfs/code_source_pytorch/FlowNetPytorch/run_inference.py:125: FutureWarning: You are using `torch.load`
with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default
value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary
objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't
have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

  network_data = torch.load(args.pretrained, map_location=torch.device('cpu'))
=> using pre-trained model 'flownets'

0%|          | 0/1 [00:00<?, ?it/s]torch.Size([3, 1800, 2880])
100%|██████████| 1/1 [00:01<00:00, 1.95s/it]"
```

```
"=> will save raw output and RGB visualization  
=> fetching img pairs in '/scratch/poupeaup-nfs/images_tests/emtest'  
=> will save everything to /scratch/poupeaup-nfs/images_tests/emtest/flow  
1 samples found  
  
/scratch/poupeaup-nfs/code_source_pytorch/FlowNetPytorch/run_inferencegpu.py:125: FutureWarning: You are using  
`torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to  
construct malicious pickle data which will execute arbitrary code during unpickling (See  
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default  
value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary  
objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via  
`torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't  
have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.  
  
network_data = torch.load(args.pretrained)  
=> using pre-trained model 'flownets'  
  
0%|  
100%| | 0/1 [00:00<?, ?it/s]torch.Size([3, 1800, 2880])  
1/1 [00:01<00:00, 1.85s/it]"
```

Désactiver l'environnement «venv-flownet» : (depuis /scratch/<username>-nfs/code_source_pytorch)

```
$ deactivate
```

- depuis /scratch/<username>-nfs :
cp images_tests/emtest/flow/imagetest_flow.png /nfs/users/<username>-nfs/
- en local :
scp front.mono.lip6.ext:~/imagetest_flow.png /Users/<le-chemin-en-local>

III. Visualisation du flot optique grâce à des vecteurs

Une fois l'inférence lancée sur des paires d'images, il peut être intéressant de visualiser le flot optique à l'aide de vecteurs. Pour cela, il y'a le fichier visu_vect.py dans le dossier « /scratch/<username>-nfs/code_source_pytorch/FlowNetPytorch ». On peut lancer la commande suivante sur le dossier « emtest » (depuis /scratch/<username>-nfs/code_source_pytorch) :

```
python3 FlowNetPytorch/visu_vect.py -f /scratch/<username>-nfs/images_tests/emtest -b 145 -e 178 --arrow_size 1 --arrow_segmentation 2
```

Remarques :

- **attention cela implique d'avoir lancé run_inference.py avant en mode « raw » sur le dossier emtest**
 - **il est nécessaire que les paires d'images du dossier « emtest » aient nommées de la manière suivante avant l'inférence : paire1_1.png, paire1_2.png, paire2_1.png, paire2_2.png, paire3_1.png...**
 - si aucune flèches jaunes n'apparaît sur les images en sortie, il faut augmenter --arrow_size
-
- L'option obligatoire -f (--folder) permet de spécifier le chemin du dossier contenant les paires d'images qui ont été traitées par l'inférence en mode « viz »
 - L'option -b (--begin_frame) (respectivement -e (--end_frame)) permet de spécifier la première paire d'images à traiter (resp la dernière)
 - L'option --arrow_size permet de régler la taille des flèches sur les images en sorties (prend des valeurs décimales).
 - L'option --arrow_segmentation (prend que des entiers) permet de régler l'espacement entre chaque flèche. Plus la valeur est grande, plus les flèches seront espacées (moins il y'en aura dans l'image)

IV. Génération d'un dataset de synthèse

Le fichier « gen_met.py » permet de générer un dataset de synthèse à partir de 2 banques de données : « bg » et « motif ». Ces 2 dossiers peuvent être téléchargés ici :

https://drive.google.com/drive/folders/1CNDzLag8ebT544ORGqUVjLsHP-4Jb_Mz?usp=share_link

Ces derniers doivent être placés au même endroit que gen_met.py.

Pour lancer la génération (de 1000 images par exemple), il suffit de lancer :

```
python3 FlowNetPytorch/gen_met.py --nb_paires 1000
```

Une fois la génération terminée, le dataset de synthèse est stocké dans « dataset_syn ». Dans celui-ci on retrouve chaque pair d'images générées dans « clean » et le flot correspondant à chaque paire dans « flow ».

V. Entraînement à partir de l'architecture « mpi_sintel_clean »

L'entraînement sert à créer son propre modèle avec ses propres poids à partir d'un jeu de données d'entraînement. Ce modèle sera ensuite utilisé de la même manière que « flownets_EPE1.951.pth » qu'on retrouve dans les paramètres de « run_inference.py ». Pour effectuer l'entraînement, on supposera que le jeu de données utilisé respecte l'arborescence du dataset « mpi_sintel_clean » c'est-à-dire que le dataset soit de la forme :

```
dataset_syn
├── clean
│   ├── pair0
│   │   ├── frame_0001.png
│   │   └── frame_0002.png
│   ├── pair1
│   │   ├── frame_0001.png
│   │   └── frame_0002.png
│   ├── pair2
│   │   ├── frame_0001.png
│   │   └── frame_0002.png
│   ├── pair3
│   │   ├── frame_0001.png
│   │   └── frame_0002.png
│   └── pair4
│       ├── frame_0001.png
│       └── frame_0002.png
└── flow
    ├── pair0
    │   └── frame_0001.flo
    ├── pair1
    │   └── frame_0001.flo
    ├── pair2
    │   └── frame_0001.flo
    ├── pair3
    │   └── frame_0001.flo
    └── pair4
        └── frame_0001.flo
```

On peut ainsi lancer l'entraînement sur le dataset « dataset_syn » de la manière suivante :

```
python3 /scratch/poupeaup-nfs/code_source_pytorch/FlowNetPytorch/main.py
/scratch/poupeaup-nfs/gen_dataset/dataset_syn -b8 -j8 -a flownets --dataset
mpi_sintel_clean --epochs 60 --split-value 0.95
```

- “/scratch/poupeaup-nfs/gen_dataset/dataset_syn” est le chemin du dataset utilisé
- -b8 indique que le nombre de paires d'images utilisées pour un batch (1 batch $\simeq$ 1 unité d'apprentissage) est de 8.
- -j8 indique que le nombre de « travailleurs » (lors de la présence de gpu) est de 8 (utile pour paralléliser l'apprentissage).
- -a flownets qu'on utilise FlowNetSimple pour l'apprentissage (et pas FlowNetCorr ou d'autres architectures par exemple)
- --dataset mpi_sintel_clean indique qu'on utilise un dataset utilisant la même arborescence que « mpi_sintel_clean »
- --epochs 60 initialise le nombre d'epochs à 60. L'apprentissage sera donc effectué 60 fois de suite sur le même dataset.

